

Aktivni Zaedno — Complete Concept

October 9, 2026 · Tony Naydenov · A platform for mass sport and sports communities (Bulgaria)

Vision and problem

"Aktivni Zaedno" ("Active Together") is a Bulgarian platform that connects people around physical activity — any sport, any city, any age — and a practical tool for the national "Mass Sport for All Ages" policy.

The infrastructure exists — gyms, pitches, parks — but there is no digital layer that coordinates the people who want to use it together:

- **I have no one to play with.** People cannot find partners outside their circle of friends.
- **I don't know who organizes what.** Initiatives are announced in closed groups, so outsiders never find them.
- **There is no data.** Nobody knows how many people play sport as amateurs, which sports are popular, or which regions are "white spots".

The idea originates from the "Soccer Planner" workshop (SoftUni, course "Full Stack Apps with AI") — an app for pickup football matches between friends — and was expanded into a multi-sport civic platform. The original intent is to find a partner for any activity: jogging, basketball on a school court, football, tennis, badminton between apartment blocks.

Roles and core objects

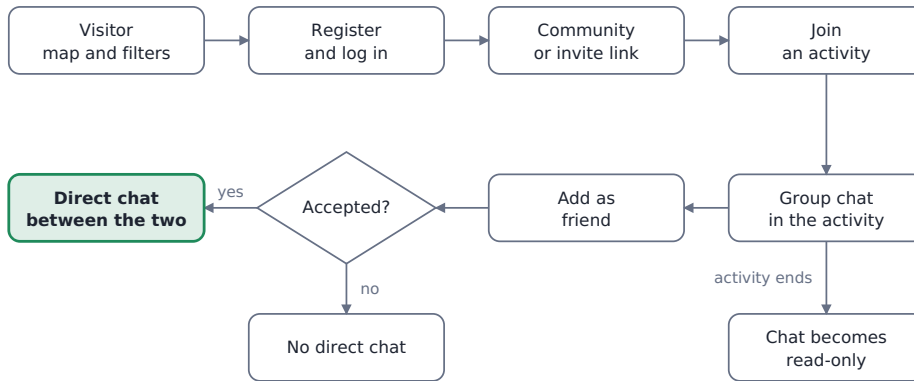
The platform is built around communities in which initiatives are organized; around them sit the venue registry, the chat and friendships.

- **Activity** (initiative) — a sports event (match, workout, run, walk) with sport type, title, description, date, time, location (a venue from the registry or a free-form place, with GPS coordinates), capacity, level (beginner, amateur, advanced) and visibility (public or closed).
- **Sports community** — a public or closed group with organizers and members; a closed one is joined through an invite link.
- **Venue** — a registry entry on the map: school, municipal or private pitch, park; supported sports, free or paid. An activity can point to a venue or to an arbitrary place.
- **Group chat** — communication between the participants of an activity, active until the activity ends.
- **Friendship** — a link between two participants; once accepted, it unlocks a direct chat.

Role	Permissions
Visitor	Sees the home page, the public map and public activities; can register
User	Manages own profile, creates communities, joins existing ones
Community member	Browses activities, joins and leaves them, takes part in the chat
Organizer	Creates and manages activities, invites members, promotes and removes members
Administrator (optional)	Manages all users, communities, activities and venues

An activity's status is upcoming, ongoing (for 2 hours from its start time), finished or canceled. An activity is active if it is upcoming or ongoing and not canceled. Members are not blocked when capacity is full — they decide for themselves. When joining, a member can request extra spots (+1, +2, +3) for friends.

User journey



The direct chat unlocks only after a friend request is accepted.

A visitor browses the map without registering; after logging in, they join a community and an activity, which gives access to the group chat. The "Add as friend" button is visible in the chat while the activity is active; only an accepted request unlocks a direct chat.

Functionality

The web app is the primary one and carries the full functionality; the mobile app is a complement with a limited scope.

Web (first version)

- Home page with live statistics (active activities, participants, sports, cities) and a public interactive map (Leaflet.js), available without registration.
- Search and filters by sport, city, date and level; clicking a marker shows details and a join button.
- **Venue registry:** a map and list of venues (school, municipal, private, parks) with supported sports; users add venues and an administrator verifies them; when a free-form place is entered, the app suggests nearby venues to avoid duplicates.
- Registration and login (email and password), a profile with name, photo, preferred sports and city.
- Communities: create, edit, delete, invite links (single-use, with a unique code), promote and remove members, leave.
- Activities: create, edit, cancel and delete by an organizer; join, leave and extra spots by members.
- Dashboard with active activities and an archive; sharing a link to an activity.
- **Group chat** in the activity, replacing the workshop's comments.
- **Friendships and direct chat:** an "Add as friend" button in the chat of an active activity; the direct chat unlocks once the friendship is accepted.

Mobile (Expo)

- Login and registration, a list of active activities, details, join and leave, extra spots, chat.

Out of scope for the first version

- Participant rating and reputation, push notifications, European expansion.

Data

The schema has ten tables: the seven from the workshop (with `activity_comments` replaced by the chat), plus the venue registry, friendships and direct messages. Every change goes through Drizzle Kit migrations.

Table	Fields	Description
users	id, email, passwordHash, name, photoUrl, city, createdAt	Users
communities	id, title, description, sport, isPublic, createdAt	Sports communities
community_members	id, communityId, userId, isOrganizer, joinedAt	Membership
community_invites	id, communityId, code, usedAt, usedByUserId	Single-use invites
venues	id, name, type (school, municipal, private, park, other), city, address, lat, lng, sports, isFree, status (pending, verified), createdByUserId, createdAt	Venue registry; new entries stay pending until verified
activities	id, communityId, venueId (optional), sport, title, description, date, location, lat, lng, capacity, level, isPublic, isCanceled	Activities; when venueId is empty, the location is a free-form place
activity_joins	id, activityId, userId, extraSlots, joinedAt	Participation and extra spots
activity_chat_messages	id, activityId, senderId, text, createdAt	Group chat, replaces comments
friendships	id, userAId, userBId, status (pending, accepted, declined), originActivityId, requestedAt, acceptedAt	Friendships, with the activity they originate from
direct_messages	id, friendshipId, senderId, text, createdAt	Direct chat, only when <code>accepted</code>

An activity is accessible only to members of its community unless it is public. The activity chat is locked (read-only) once the activity becomes finished or canceled.

Venue duplication. When a user enters a free-form place, the app suggests nearby venues from the registry and lets the place be saved as a new venue; before entering the registry, a new venue is checked for nearby duplicates and stays `pending` until verified.

Technology and architecture

The stack is TypeScript throughout, in a single Node.js monorepo, so types are shared between the web and mobile apps.

Layer	Choice
Backend and frontend	Next.js, React, TypeScript, Tailwind CSS
Database	Neon (serverless PostgreSQL) and Drizzle ORM
Map	Leaflet.js
Mobile	React Native and Expo
Files and photos	Cloudflare R2
Deploy	Vercel (web and Expo web export); optionally an Android APK in GitHub Releases

Monorepo structure

- `az-web/` — Next.js app: `src/app/` (frontend), `src/app/api/` (REST endpoints), `src/services/` (business logic), `src/db/` (schema), `src/drizzle/` (migrations)
- `az-mobile/` — Expo app
- `az-shared/` — shared TypeScript types (User, Activity, Message...)
- three `AGENTS.md` files (root, `az-web`, `az-mobile`) with guidance for AI agents

Principles

- Business logic lives in a service layer used by both Server Actions (web) and the RESTful API (mobile).
- Server-side paging everywhere and database indexes from the start.
- Auth: passwords hashed with bcrypt or argon2; JWT with a random `JWT_SECRET`; cookies for web, Bearer header for the API.
- Server components by default; client components only where interactivity is needed.
- The chat needs real-time messages; the mechanism (polling, SSE or an external WebSocket service) is an open question, because Vercel is serverless and does not hold persistent WebSocket connections.

API for the mobile app

A minimal RESTful API in the Next.js project; everything except login, registration and the documentation requires a JWT in the Bearer header.

Endpoint	Method	Description
/api/auth/login	POST	Log in with email and password, returns a JWT
/api/auth/register	POST	Register (email, password, name)
/api/activities	GET	Active activities, paged
/api/activities/[id]	GET	Details, participants, chat
/api/activities/[id]/join	POST	Join
/api/activities/[id]/leave	POST	Leave
/api/activities/[id]/slots	POST	Extra spots
/api/activities/[id]/messages	GET, POST	Read and send chat messages (replaces /comments)
/api/venues	GET, POST	Venues by city, sport and map bounds; add a new venue
/api/venues/[id]	GET	Venue details and its upcoming activities
/api/friendships	GET, POST	Friend requests and friends list
/api/friendships/[id]	PATCH	Accept or decline a request
/api/friendships/[id]/messages	GET, POST	Direct chat (only when accepted)
/api/docs	GET	API documentation as HTML

Strategy

"Aktivni Zaedno" is a civic initiative with zero state budget: it seeks recognition, not funding. The pitch document targets the Ministry of Youth and Sports and cites the "Mass Sport for All Ages" policy.

The ask to institutions

- A public recommendation of the platform by the ministry.
- A mention in official communications and campaigns for an active lifestyle.
- An endorsement letter — key for future grants under EU sport and digitalization programs.
- A link to municipalities, through local sports coordinators.

Rollout

1. Pilot in the regional capital cities of Bulgaria.
2. The whole country — 265 municipalities.
3. Europe: compatriots on holiday or a business trip find a partner to train with.

Added value: as a side effect, the platform creates the first national data on amateur sport activity — how many people play, which sports are popular and where the "white spots" are.

The business model is still open; for now the goal is recognition and proven usage, followed by grants, municipal partnerships or other sources.

Development order for Claude Code

Work proceeds in small steps, with a commit and push to GitHub after each successful implementation.

1. Monorepo `aktivni-zaedno` with `az-web`, `az-mobile`, `az-shared`; Git repo, `.gitignore` and the three `AGENTS.md` files.
2. Neon database `AktivniZaednoDB`, Drizzle schema and migrations for ten tables, a seed script (`npm run db:seed`) with sample users, communities, venues and activities.
3. Public pages: home page with statistics and map, login, registration, shared layout.
4. Auth with cookies and JWT, bcrypt, Server Actions; protected routes.
5. Dashboard (`/dashboard`) and the activity page with join, leave and extra spots.
6. Communities: list, details, create, edit, delete, invite links, member management.
7. Venue registry (venues): list and map, user submissions, administrator verification and nearby-venue suggestions against duplicates; then creation and management of activities by organizers, with a choice of venue or free-form place.
8. Public map (Leaflet.js) with filters and live statistics.
9. **Group chat** in the activity, replacing comments.

10. **Friendships and direct chat.**

11. RESTful API and the Expo app: login, dashboard, details, chat.

12. Deploy to Vercel, a performance test seeded with 500 communities, 5,000 activities and 3,000 users; indexes and paging.

The demo account for testing is `demo / demo123`.

Open questions

- Real-time chat on Vercel (serverless): polling, SSE or an external WebSocket service?
- Chat moderation and safety: reporting and blocking users before the chat is opened to strangers.
- Business model after proven usage: grants, municipalities or something else.
- When do participant ratings and reputation, and notifications, come in?
- How are new venues verified: by an administrator, by organizers, or automatically after confirmation from several users?
- The federation B2B product (membership, tournaments, reporting to the Ministry of Youth and Sports) — after the mass-sport app.